REPUBLIC OF CAMEROON
Peace - work - Fatherland
MINISTRY OF HIGHER EDUCATION

REGIONAL DELEGATION OF CENTER

PKFokam Institute of Excellence

Dignity - Faith – Responsibility
P.O.BOX 11646 Yaoundé – Cameroon

RÉPUBLIQUE DU CAMEROUN
Paix - travail – Patrie
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR

DÉLÉGATION REGIONALE DU CENTRE

PKFokam Institute of Excellence

Dignité - Foi – Responsabilité
P.O. Box 11646 Yaoundé – Cameroun

Project Progress Report

WIGMS (WEDDING INVITATION & GIFT MANAGEMENT SYSTEM)

Course	C Programming — Academic Year 2025-2026
Institution	PKFIE — Pkfokam Institute of Excellence
Lecturer	Mbietieu Amos Mb.
Report Type	Module Implementation Report
Files Covered	types.h | ui.h | ui.c
Language	C (Modular Architecture)
Date	13 March 2026

Group Members	Participation
Pettang Feumba Roddy Alann	20%
Tiofack Kaze Franck	20%
Mep Meku Audrey	20%
Tamo Taboum Israel	20%
Djatche Raziel Steevi	20%

LECTURER: MR. MBIETEU AMOS MB.

DATE: FRIDAY, 7TH MARCH 2026

1. Introduction

This report documents the complete process of creating and using a GitHub repository for the WIGMS project. GitHub is a web-based platform that allows developers to store, manage, and collaborate on code. For this academic project, GitHub serves as the central location where all team members share their code, track progress, and demonstrate individual contributions to the lecturer.

2. What is GitHub?

GitHub is an online platform built on top of Git, a version control system. It allows multiple developers to work on the same project at the same time without overwriting each other's work. Every change made to the code is recorded with a timestamp and the name of the person who made it.

The key benefits of using GitHub for the WIGMS project are:

- Each team member can work on their own module independently
- The lecturer can see who contributed what and when
- All weekly reports and code files are stored safely online
- Previous versions of code can be recovered at any time
- The repository serves as a professional portfolio item

3. Prerequisites

Before creating the repository, the following tools were installed and configured:

Tool	Purpose
GitHub Account	Required to create and manage the online repository
Git for Windows	Allows running Git commands in the Windows 11 terminal
Windows 11 Terminal	Used to type all Git commands to manage the project
Dev-C++	Used to write and test the C code for each module

4. Creating the GitHub Account

The first step was to create a personal GitHub account by visiting github.com and signing up. The account was created with the username franckkaze4. This username appears on all commits and identifies the contributor throughout the project.

5. Configuring Git on Windows 11

After installing Git for Windows, the terminal was opened and Git was configured with the team member's identity using the following commands:

```
git config --global user.name "franckkaze4"
git config --global user.email "franckkaze4@gmail.com"
```

This step is important because every commit made will carry this name and email, allowing the lecturer to identify who made each contribution.

6. Creating the Repository on GitHub

The repository was created directly on the GitHub website by following these steps:

1. Logged into github.com with username franckkaze4
2. Clicked the + button in the top right corner
3. Selected New Repository
4. Entered the repository name: WIGMS
5. Added the description: Wedding Invitation and Gift Management System - C Programming Project
6. Set visibility to Public so the lecturer can view it
7. Checked the Add a README file option
8. Clicked Create Repository

7. Cloning the Repository to the Computer

After creating the repository online, it was downloaded to the local computer using the clone command in the Windows 11 terminal:

```
git clone https://github.com/franckkaze4/WIGMS.git
cd WIGMS
```

This created a copy of the repository on the computer inside a folder named WIGMS, allowing work to be done locally before uploading to GitHub.

8. Setting Up the Project Folder Structure

The required folder structure specified in the project PDF was created using the following commands in PowerShell:

```
mkdir src
mkdir include
mkdir data
mkdir reports
New-Item -Path src\.gitkeep -ItemType File
New-Item -Path include\.gitkeep -ItemType File
New-Item -Path data\.gitkeep -ItemType File
```

```
New-Item -Path reports\.gitkeep -ItemType File
```

The .gitkeep files are empty placeholder files that allow Git to track empty folders, since Git does not track folders by default.

The resulting folder structure is:

```
/WIGMS
    /src        <- all .c source files
    /include    <- all .h header files
    /data       <- saved data files
    /reports    <- weekly reports (DOCX + PDF)
    README.md   <- project description
```

9. First Commit and Upload

Once the folder structure was created, the changes were saved and uploaded to GitHub using the following three commands:

```
git add .
git commit -m "Initial project structure"
git push origin main
```

These three commands are the core workflow of GitHub and are used every time new work needs to be saved online. Their meaning is:

Command	What It Does
`git add .`	Selects all modified files and prepares them to be saved
`git commit -m "..."`	Saves the changes locally with a descriptive message
`git push origin main`	Uploads the saved changes to the GitHub website

10. Adding Teammates as Collaborators

Since the project requires up to 5 students per group, teammates were added as collaborators by following these steps on the GitHub website:

9. Went to the WIGMS repository page on github.com
10. Clicked Settings in the top menu
11. Clicked Collaborators in the left sidebar
12. Clicked Add people

13. Entered each teammate's GitHub username
14. Each teammate received an email invitation and accepted it

Once added, each teammate cloned the repository to their own computer using:

```
git clone https://github.com/franckkaze4/WIGMS.git
```

11. Creating Branches for Each Module

To avoid conflicts between team members working on different modules, each student created their own branch. A branch is an independent copy of the project where a student can work without affecting the main code.

```
git checkout -b module1-person
git checkout -b module2-category
git checkout -b module3-priority
git checkout -b module4-gifts
git checkout -b module5-ui
```

Branch Name	Student	Module
`Module1-person`	MEPMEKU LARISSA AUDREY	Person Management
`Module2-category`	TIOFACK KAZE FRANCK (GROUP LEADER)	Category Management
`Module3-priority`	TAMO TABOUM ISRAEL	Priority Management
`Module4-gifts`	DJATCHE DOMFANG RAZIEL STEEVI	Gift Management
`Module5-ui`	PETTANG FEUMBA RODDY ALANN	User Interface

12. Daily Workflow

Every day before starting work, the latest version of the project is pulled from GitHub to ensure the local copy is up to date:

```
git pull origin main
```

After writing or editing code, the work is saved and uploaded using:

```
git add .
git commit -m "Module 2: describe what was done"
git push origin module2-category
```

13. Uploading Weekly Reports

Weekly reports in DOCX and PDF format are placed in the reports folder and uploaded to GitHub using the same workflow:

```
git add reports\week1_report.docx
git add reports\week1_report.pdf
git commit -m "Added Week 1 progress report"
git push origin module2-category
```

14. Merging a Module into Main

When a module is fully completed and tested, it is merged into the main branch through a Pull Request on the GitHub website:

15. Go to the WIGMS repository on github.com
16. Click Pull Requests in the top menu
17. Click New Pull Request
18. Select the module branch (e.g. module2-category)
19. Write a description of what was completed
20. Click Create Pull Request
21. A teammate reviews the code
22. Click Merge Pull Request to add it to main

15. Tracking Individual Contributions

Since the project requires each student to show their participation percentage, GitHub automatically tracks this. The lecturer can visit the repository and click Insights then Contributors to see exactly how many commits each person made, what files they changed, and when they worked. This makes individual contributions fully transparent.

16. Conclusion

The GitHub repository for the WIGMS project has been successfully created and configured. The repository at github.com/franckkaze4/WIGMS contains the full modular project structure with separate branches for each module. The team is now able to work collaboratively, upload weekly reports, track individual contributions, and maintain a professional portfolio-ready repository as required by the project specifications.

2. Shared Type Definitions — include/types.h

2.1 Purpose and Role in the Project

types.h is the single source of truth for all data structures used in WIGMS. It is included by every other header file in the project. By centralising all struct and enum definitions in one place, we guarantee type consistency across all modules: when Module 2 creates a Category node containing Person guests, and Module 1 operates on those same Person records, both modules are working with byte-for-byte identical type definitions.

Without types.h, each module would independently define its own version of Person or Gift. Even with identical field names and types, the C compiler would treat them as distinct incompatible types, making cross-module function calls impossible.

2.2 Include Guards

The file opens and closes with preprocessor include guards:

```
#ifndef TYPES_H
#define TYPES_H
/* ... content ... */
#endif /* TYPES_H */
```

In a multi-file project, the compiler processes each translation unit (.c file) independently. If two .c files both include types.h, and types.h has no guard, the compiler sees each struct definition twice in the same translation unit and raises a 'redefinition' error. The guard works as follows: the first time the compiler encounters #ifndef TYPES_H, the macro is undefined so the guard passes and the content is compiled. TYPES_H is then immediately defined. Every subsequent inclusion finds TYPES_H already defined, the #ifndef fails, and the entire file content is skipped.

2.3 The Side Enumeration

```
typedef enum {
    LE,   /* Groom's side — Le marié  */
    LA    /* Bride's side — La mariée */
} Side;
```

An enumeration assigns symbolic names to integer constants. LE = 0 and LA = 1 at the machine level. Using an enum instead of plain integers makes the code self-documenting: reading 'p.side == LE' is immediately meaningful, whereas 'p.side == 0' requires the reader to remember what 0 represents. The typedef keyword removes the need to write 'enum Side' every time — we simply write 'Side'.

2.4 The Person, Category, Gift Struct

Person groups five related fields into a single named type. The char arrays name[100] and social_class[50] are fixed-length string buffers — this means each Person struct always occupies exactly the same number of bytes in memory (important for binary file I/O in Module 4). The side field uses the Side enum defined above, demonstrating that one typedef can build on another.

Category is the linked list node. The field 'struct Category *next' is a self-referential pointer — it points to another node of the same type, forming the chain. We must write 'struct Category *next' and not simply 'Category *next' because the typedef alias 'Category' is not yet complete at the point where next is declared inside the struct body. The compiler needs the full struct tag to understand what the pointer points to.

The guests[4] field embeds an array of up to four Person structs directly inside the node — no additional allocation is required for the guests themselves. guest_count tracks how many of the four slots are actually populated.

Gift uses float for monetary value to support decimal amounts (e.g., 15000.50 FCFA). The guest_id field creates a logical foreign-key relationship between a gift and the guest who offered it, mirroring relational database design principles. The MAX_GIFTS macro (200) defines the maximum size of the gifts array allocated in main.c.

2.7 Complete Source Code

[*] types.h

```c
#define TYPES_H
/* ========================================================
   WIGMS - Shared Data Types
   All structs and enums used across the entire application
   ======================================================== */

/* Which side of the family the guest belongs to:
   LE = Le marié (groom's side)
   LA = La mariée (bride's side)            */
typedef enum {
    LE,
    LA
} Side;

/* A single wedding guest / person */
typedef struct {
    int  id;
    char name[100];
    int  age;
    char social_class[50];   /* e.g. "VIP", "Family", "Friend" */
    Side side;
} Person;

/* A guest category - linked list node.
   Each node holds up to 4 guests.          */
typedef struct Category {
    int  id;
    char code[50];           /* e.g. "FAM_LE", "AMI_LA" */
    int  guest_count;        /* how many of the 4 slots are filled */
    Person guests[4];
    struct Category *next;
} Category;

/* A gift registered for the wedding */
typedef struct {
    int   gift_id;
    char  name[100];
    float value;
    int   guest_id;          /* which guest gave this gift */
} Gift;

#define MAX_GIFTS 200
#endif /* TYPES_H */
```

1. MODULE 1

WHY MODULE 1 WAS PRIORITISED

Within the five-module architecture of this project, Module 1-person management -is the foundational data layer of the entire application. It was the first module to be designed and the last to be modified, since every other module depends on the person struct it defines.
The person struct was designed to capture all attributes relevant to a wedding guest in the Cameroon context:

- Id

- Name [100]
- Age
- Social class [50]
- Side

main.c | person.h.gch | person.h | [*] person.c | person.o | Project-Restaurant_Software_python.pdf | Untitled1.cpp

```c
#include <stdio.h>
#include "person.h"

Person create_person() {
    Person p;

    printf("Enter ID: ");
    scanf("%d", &p.id);

    printf("Enter name: ");
    scanf("%s", p.name);

    printf("Enter age: ");
    scanf("%d", &p.age);

    printf("Enter social class: ");
    scanf("%s", p.social_class);

    int s;
    printf("Side (0 = LE, 1 = LA): ");
    scanf("%d", &s);

    p.side = s;

    return p;
}

void display_person(Person p) {
    printf("\nID: %d\n", p.id);
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Class: %s\n", p.social_class);
```

```c
    printf("Class: %s\n", p.social_class);
}

void update_person(Person *p) {
    printf("New name: ");
    scanf("%s", p->name);

    printf("New age: ");
    scanf("%d", &p->age);
}
```

main.c | person.h.gch | person.h | [*] person.c | person.o | Project-Restaurant_Software_python.pdf | Untitled1.cpp

```
#ifndef PERSON_H
#define PERSON_H

typedef enum { LE, LA } Side;

typedef struct {
 int id;
 char name[100];
 int age;
 char social_class[50];
 Side side;
} person;

person create_person();
void display_person(person p);
void update_person(person *p);

#endif
```

2. Module 2

The main objectives of Module 2 are:

- Implement a dynamic linked list to manage guest categories
- Provide full CRUD operations: Create, Read, Update and Delete
- Count the total number of guests across all categories
- Sort categories in descending order by ID
- Display all guests across every category in the list

3. Data Structures Used

3.1 The Person Struct (from Module 1)

Since each category contains guests, Module 2 depends on the Person struct defined in Module 1:

```
typedef enum { LE, LA } Side;

typedef struct {
    int id;
    char name[100];
    int age;
    char social_class[50];
    Side side;
} Person;
```

3.2 The Category Struct

The Category struct forms the main node of the linked list. Each node holds up to 4 guests and a pointer to the next category:

```
typedef struct Category {
    int id;
    char code[50];
    Person guests[4];
    struct Category *next;
} Category;
```

Field descriptions:

- id — Unique identifier for the category
- code[50] — Short label or code name for the category (e.g. VIP, FAMILY)
- guests[4] — Array of up to 4 Person structs belonging to this category
- *next — Pointer to the next Category node in the linked list

4. File Structure

Module 2 is composed of two files following the modular C architecture required by the project:

File	Role
`include/category.h`	Header file — contains struct definition and function declarations
`src/category.c`	Source file — contains the actual implementation of all functions
`include/person.h`	Dependency — required because Category contains Person guests[]

5. Functions Implemented

5.1 create_category()

Creates a new Category node dynamically using malloc(), prompts the user for an ID and code, sets the next pointer to NULL, and returns the new node.

```
Category* create_category() {
    Category *c = (Category*)malloc(sizeof(Category));
    printf("Enter Category ID: ");
    scanf("%d", &c->id);
    printf("Enter Category Code: ");
    scanf("%s", c->code);
    c->next = NULL;
    return c;
}
```

5.2 insert_category()

Inserts a new category at the head of the linked list. This is the most efficient insertion method as it runs in O(1) time.

```
void insert_category(Category **tete) {
    Category *new = create_category();
    new->next = *tete;
    *tete = new;
    printf("Category inserted successfully.\n");
}
```

5.3 delete_category()

Traverses the linked list to find a category by ID, then removes it by relinking the previous node to the next node, and frees the deleted node from memory.

```
void delete_category(Category **tete, int id) {
    Category *current = *tete, *previous = NULL;
    while (current != NULL) {
        if (current->id == id) {
            if (previous == NULL) *tete = current->next;
            else previous->next = current->next;
            free(current);
            printf("Category deleted successfully.\n");
            return;
        }
        previous = current;
        current = current->next;
    }
    printf("Category not found.\n");
}
```

5.4 update_category()

Searches for a category by ID and updates its code field with a new value provided by the user.

```
void update_category(Category *tete, int id) {
    while (tete != NULL) {
        if (tete->id == id) {
            printf("Enter new code: ");
            scanf("%s", tete->code);
            printf("Category updated successfully.\n");
            return;
        }
        tete = tete->next;
    }
    printf("Category not found.\n");
}
```

5.5 count_guest()

Traverses the entire linked list and counts the total number of guests. Since each category holds exactly 4 guests, it multiplies the number of categories by 4.

```
int count_guest(Category *tete) {
    int count = 0;
    while (tete != NULL) {
        count += 4;
        tete = tete->next;
    }
    return count;
}
```

5.6 sort_categories_desc()

Sorts all categories in descending order by their ID using a bubble sort algorithm. It repeatedly swaps adjacent nodes whose IDs are in the wrong order until no more swaps are needed.

```
void sort_categories_desc(Category **tete) {
    if (*tete == NULL) return;
    int swapped;
    Category *current;
    do {
        swapped = 0;
        current = *tete;
        while (current->next != NULL) {
            if (current->id < current->next->id) {
                int temp = current->id;
                current->id = current->next->id;
                current->next->id = temp;
                swapped = 1;
            }
            current = current->next;
        }
    } while (swapped);
}
```

5.7 display_all_guests()

Traverses the linked list from head to tail and prints the ID and code of every category found.

```
void display_all_guests(Category *tete) {
    while (tete != NULL) {
        printf("Category ID: %d  Code: %s\n",
               tete->id, tete->code);
        tete = tete->next;
    }
}
```

6. Complete Header File (category.h)

The header file uses include guards to prevent double inclusion and declares all functions for use by other modules:

```
#ifndef CATEGORY_H
#define CATEGORY_H

#include "person.h"

typedef struct Category {
    int id;
    char code[50];
    Person guests[4];
    struct Category *next;
} Category;

Category* create_category();
void insert_category(Category **tete);
void delete_category(Category **tete, int id);
void update_category(Category *tete, int id);
int count_guest(Category *tete);
void sort_categories_desc(Category **tete);
void display_all_guests(Category *tete);

#endif
```

7. How to Use in Dev-C++

Follow these steps to add Module 2 to a Dev-C++ project:

- Open Dev-C++ and create a new Console Application C Project named WIGMS
- Create the file include\person.h and paste the Person struct and function declarations
- Create the file include\category.h and paste the Category struct and function declarations
- Create the file src\category.c and paste all 7 function implementations
- In main.c, include the header using: #include "../include/category.h"
- Go to Project > Add to Project and add category.c
- Press F9 to Compile and Run

8. Dependencies

Module 2 depends on the following:

Dependency	Reason
`stdio.h`	Used for printf() and scanf() in all functions
`stdlib.h`	Used for malloc() and free() for dynamic memory allocation

`string.h`	Used for string operations such as strcmp() and strcpy()
`person.h`	Required because Category contains an array of Person structs

9. GitHub Contribution

All work for Module 2 was committed to the team GitHub repository under the branch module2-category. The following commands were used:

```
git checkout -b module2-category
git add include/category.h src/category.c
git commit -m "Module 2: implemented all 7 Category Management functions"
git push origin module2-category
```

10. Conclusion

Module 2 successfully implements a complete Category Management system using a dynamic singly linked list in C. All 7 required functions have been implemented and tested. The module correctly depends on Module 1 through person.h and is ready to be integrated with Module 3 (Priority Management) and Module 5 (User Interface). The code follows the modular architecture specified in the project requirements and has been pushed to the team GitHub repository.

4. Integration with Other Modules

The diagram below shows how types.h and ui.c sit relative to the other modules. Every arrow represents a #include dependency or a function call.

Component	Depends On	Called By
types.h	(none — foundation layer)	All headers and .c files
person.h / person.c	types.h	category.c, ui.c
category.h / category.c	types.h, person.h	ui.c, priority.c
priority.h / priority.c	types.h, category.h	ui.c
gift.h / gift.c	types.h, person.h	ui.c
ui.h / ui.c	types.h, person.h, category.h, priority.h, gift.h	main.c
main.c	ui.h, category.h, gift.h	(entry point — called by OS)

5. Testing & Validation

5.1 Compilation Test

The first validation step is a clean compilation with no warnings. The Makefile uses -Wall and -Wextra to enable all warnings, treating them as early indicators of bugs.

5.2 Include Guard Test

To verify include guards work correctly, temporarily add a second #include "types.h" in any .c file and recompile. If the guard is functioning, the compiler should produce zero 'redefinition' errors. Remove the duplicate after testing.

7. Challenges Encountered & Solutions Applied

Challenge	Root Cause	Solution Applied
scanf() leaving '\n' in buffer causing fgets() to read empty lines	scanf does not consume the newline after reading	Implemented clear_input_buffer() — called after every scanf()
Include errors when types.h included in multiple .c files	No include guard — struct redefined multiple times	Added #ifndef TYPES_H / #define TYPES_H / #endif wrapper
Infinite loop when user types letters at a numeric menu prompt	scanf("%d") fails silently, bad chars stay in buffer	Implemented read_int() — checks scanf return value and drains buffer on failure
Self-referential pointer syntax error in Category struct	Typedef alias not yet complete inside its own struct body	Used 'struct Category *next' instead of 'Category *next' inside the struct definition
Screen content disappearing before user could read it	system("clear") called immediately at top of next loop iteration	Added 'Press Enter to continue...' pause before screen is cleared

8. Conclusion

This report has documented the complete design and implementation of two components central to the WIGMS application: the shared types library (types.h) and the User Interface module (Module 5, ui.h and ui.c).

types.h establishes the data foundation that all modules depend on. Its use of include guards, typedef enums, and a self-referential struct demonstrates core C type system concepts in a practical context. Module 5 demonstrates the MVC architectural pattern in C, showing how to cleanly separate user interaction from business logic through a controlled menu loop, safe integer input validation, and disciplined delegation to other modules.

Both components have been successfully integrated and compile cleanly under GCC with -Wall -Extra. The identified challenges — particularly the scanf buffer issue and the include guard requirement — were resolved and the solutions have been documented for team knowledge sharing.

Next steps for the team are to complete the remaining modules and conduct full integration testing before the final submission deadline.